Welcome

Digital and Computer Vision

Lesson 4: Session 2

Face Recognition with OpenCV & Deep Learning

Summary of the previous session: Introduction to Deep Learning & CNNs for Image Processing

In the previous session we have covered :

Introduction to Deep Learning
Overview of Convolutional Neural Networks (CNNs).
Using TensorFlow/Keras with OpenCV.

Summary of the current session: Face Recognition with OpenCV & Deep Learning

In this session we will covered the following:

Understanding FaceNet and Dlib face recognition models.
Real-time face recognition using OpenCV

Exercise from previous session 1/7

The last proposed code Real-Time Person detection with Use SSD Mobilenet V2 was rather slow. Propose a way (and the code) to improve the speed having in mind that we need only a person detection.

Key optimizations to make faster person detection:

1. Filter for "person" only. The SSD MobileNet model detects 90+ COCO classes. If you're only interested in person (class_id = 1), then skip all other detections:
Update inside the loop:

```
for i in range(len(scores)):
    if scores[i] < 0.5 or classes[i] != 1:
        Continue
```

NB ! This didn't speed up the detection process, but only skip drawing boxes.

Exercise from previous session 2/7

2. Use GPU. If your system has a CUDA-compatible GPU and TensorFlow-GPU is installed, detection will be significantly faster. You can verify with:

```
from tensorflow.python.client import device_lib
print(device_lib.list_local_devices())
```

Exercise from previous session 3/7

3. Use a smaller model TensorFlow Lite :

Download the TFLite model and label file:
Get the quantized SSD MobileNet V2 TFLite from TensorFlow's model zoo:

https://storage.googleapis.com/download.tensorflow.org/models/tflite/coco_ssd_mobilenet_v1_1.0_quant_2018_06_29.zip

Unzip and put the detect.tflite and labelmap.txt inside in your Model\TensorFlow folder.

Exercise from previous session 4/7

```
import cv2
import numpy as np
import tensorflow as tf

MODEL_PATH  = "..\Models\TensorFlow\detect.tflite"
LABEL_PATH  = "..\Models\TensorFlow\labelmap.txt"
THRESHOLD   = 0.5     # minimum confidence to show detection
PERSON_LABEL = "person"

with open(LABEL_PATH, 'r') as f:
    labels = [line.strip() for line in f.readlines()]
# The label file you downloaded  is one-based; it starts with a dummy "???"
if labels[0].startswith("???"):
    labels = labels[1:]
# find the index in the label list for "person"
try:
    person_idx = labels.index(PERSON_LABEL)
except ValueError:
    person_idx = 0  # COCO convention: class ID 1 is person if no header
print("Person ID:", person_idx)
```

Exercise from previous session 5/7

```
# Load the model
interpreter = tf.lite.Interpreter(model_path=MODEL_PATH)
interpreter.allocate_tensors()

input_details  = interpreter.get_input_details()
output_details = interpreter.get_output_details()

# model expects input size like [1, h, w, 3]
input_height = input_details[0]['shape'][1]
input_width  = input_details[0]['shape'][2]

# WEBCAM LOOP
cap = cv2.VideoCapture(0)
if not cap.isOpened():
    raise RuntimeError("Could not open webcam.")

while True:
    ret, frame = cap.read()
    if not ret:
        break
```

Exercise from previous session 6/7

```
# 1) Preprocess: resize + pad to model input, keep aspect ratio
h, w, _ = frame.shape
img = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
img_resized = cv2.resize(img, (input_width, input_height))
input_data = np.expand_dims(img_resized, axis=0)

# 2) Set the tensor (quantized model wants uint8)
if input_details[0]['dtype'] == np.uint8:
    input_data = np.uint8(input_data)
else:
    input_data = np.float32(input_data) / 255.0

interpreter.set_tensor(input_details[0]['index'], input_data)
interpreter.invoke()

# 3) Get the outputs
#    [0] = locations, [1] = classes, [2] = scores, [3] = count
boxes   = interpreter.get_tensor(output_details[0]['index'])[0]  # [N,4]
classes = interpreter.get_tensor(output_details[1]['index'])[0].astype(int)
scores  = interpreter.get_tensor(output_details[2]['index'])[0]
count   = int(interpreter.get_tensor(output_details[3]['index'])[0])
```

Exercise from previous session 7/7

```
    # 4) Draw detections for “person” only
    for i in range(count):
        if scores[i] < THRESHOLD or classes[i] != person_idx:
            continue

        # box coords are [ymin, xmin, ymax, xmax] normalized
        ymin, xmin, ymax, xmax = boxes[i]
        x1, x2 = int(xmin * w), int(xmax * w)
        y1, y2 = int(ymin * h), int(ymax * h)

        # draw a rectangle and label
        cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
        label = f"{PERSON_LABEL}: {scores[i]:.2f}"
        cv2.putText(frame, label, (x1, y1 - 10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 1)

    cv2.imshow("TFLite Person Detection", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Face Processing - Two Main Tasks

1. **Face Detection** : Find faces in an image — like we're doing now (person detection).

You can use:
Haar Cascades (classic, fast)
MTCNN (deep learning)
SSD MobileNet (we're using)

2. **Face Recognition** / Identification : Answering: "Who is this face?"

To recognize your own face, you need:
A few photos of yourself (different angles, lighting)
Optionally photos of other people (for contrast)
A model trained to match faces (not just detect them)

Easy Solution - Use face_recognition Library

There’s an awesome Python library based on deep learning (Dlib under the hood) that makes this super easy. Install this library :

pip install face_recognition

If you are missing cmake, you should install cmake first. Check with :

cmake – version in a cmp.

Download the cmake installer form :

https://cmake.org/download/

Run the installer then restart your computer.

Use face_recognition Library example code 1/3

```
import face_recognition
import cv2

# Load your face from an image
known_image = face_recognition.load_image_file("..\Images\Faces\Koko\koko_6.jpg")
known_encoding = face_recognition.face_encodings(known_image)[0]

# Start webcam
cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Resize for speed
    small_frame = cv2.resize(frame, (0, 0), fx=0.25, fy=0.25)
    rgb_small = cv2.cvtColor(small_frame, cv2.COLOR_BGR2RGB)
```

Use face_recognition Library example code 2/3

```
try:
    face_locations = face_recognition.face_locations(rgb_small)
    if face_locations:
        face_encodings = face_recognition.face_encodings(rgb_small, face_locations)
    else:
        print("No faces found in the frame.")
        face_encodings = []
except Exception as e:
    print("Error during face encoding:", str(e))

for (top, right, bottom, left), face_encoding in zip(face_locations, face_encodings):
    matches = face_recognition.compare_faces([known_encoding], face_encoding)
    name = "Unknown"

    if True in matches:
        name = "YOU"

    # Scale back up
    top *= 4
    right *= 4
    bottom *= 4
    left *= 4
```

Use face_recognition Library example code 3/3

```
        # Draw box and name
        cv2.rectangle(frame, (left, top), (right, bottom), (0, 255, 0), 2)
        cv2.putText(frame, name, (left, top - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.75, (0, 255, 0), 2)

    cv2.imshow("Face Recognition", frame)
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

cap.release()
cv2.destroyAllWindows()
```

Multi-Person Face Recognition 1/2

Organize Your Training Images. Each subfolder is a person's name, containing several face images.
Structure your dataset like this:

```
Faces/
Person1
        img1.jpg
        img2.jpg
        img3.jpg
        …
Person2
        img1.jpg
        img2.jpg
        img3.jpg
        …
Person3
        img1.jpg
        img2.jpg
        img3.jpg
        …
```

Multi-Person Face Recognition 2/2

Add 3–5 good images per person (clear, well-lit, forward-facing).

Lower tolerance (e.g., 0.4) for stricter matching, or raise it (e.g., 0.6) to allow more leniency.

You can save encodings to disk (pickle or JSON) for faster loading later.

Multi-Person Face Recognition code example 1/4

```
import face_recognition
import cv2
import os
import pickle

known_encodings = []
known_names = []

base_folder = "..\\Images\Faces\\"
for person_name in os.listdir(base_folder):
    person_folder = os.path.join(base_folder, person_name)
    if not os.path.isdir(person_folder):
        continue

    for filename in os.listdir(person_folder):
        if filename.endswith((".jpg", ".png")):
            image_path = os.path.join(person_folder, filename)
            image = face_recognition.load_image_file(image_path)
            encodings = face_recognition.face_encodings(image)
```

Multi-Person Face Recognition code example 2/4

```
        if encodings:
            known_encodings.append(encodings[0])
            known_names.append(person_name)
        else:
            print(f"[Warning] No face found in {image_path}")

# Save to a file
data = {"encodings": known_encodings, "names": known_names}
with open("..\\Models\\FaceRecognition\\face_encodings.pkl", "wb") as f:
    pickle.dump(data, f)

# Start webcam
cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Resize for speed
    small_frame = cv2.resize(frame, (0, 0), fx=0.25, fy=0.25)
    rgb_small = cv2.cvtColor(small_frame, cv2.COLOR_BGR2RGB)
```

Multi-Person Face Recognition code example 3/4

```
try:
        face_locations = face_recognition.face_locations(rgb_small)
        if face_locations:
            face_encodings = face_recognition.face_encodings(rgb_small, face_locations)
        else:
            print("No faces found in the frame.")
            face_encodings = []
    except Exception as e:
        print("Error during face encoding:", str(e))

    for (top, right, bottom, left), face_encoding in zip(face_locations, face_encodings):
        matches = face_recognition.compare_faces(known_encodings, face_encoding, tolerance=0.45)
        face_distances = face_recognition.face_distance(known_encodings, face_encoding)

        name = "Unknown"
        if matches:
            best_match_index = face_distances.argmin()
            if matches[best_match_index]:
                name = known_names[best_match_index]
```

Multi-Person Face Recognition code example 4/4

```
        # Draw box
        top *= 4
        right *= 4
        bottom *= 4
        left *= 4

        cv2.rectangle(frame, (left, top), (right, bottom), (0, 255, 0), 2)
        cv2.putText(frame, name, (left, top - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.75, (0, 255, 0), 2)
    cv2.imshow("Face Recognition", frame)
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

cap.release()
cv2.destroyAllWindows()
```

Load from .pkl

```
import pickle

with open("face_encodings.pkl", "rb") as f:
    data = pickle.load(f)

# Access as a dictionary
known_encodings = data["encodings"]
known_names = data["names"]
```

Dlib and face_recognition Library

The actual model behind face_recognition is from Dlib.

Dlib is an open-source, modern C++ toolkit with Python bindings, designed for:
Machine learning
Computer vision
Image processing
Numerical optimization

Dlib was created by Davis E. King, computer scientist and software engineer, former researcher at the U.S. Department of Defense.

You can find Dlib on GitHub: davisking/dlib

Dlib Key Concepts

Model: Dlib uses a ResNet-34 architecture. ResNet (Residual Network) is a family of deep convolutional networks introduced by Microsoft Research in 2015. Its key idea is the use of residual blocks to allow training of very deep networks.

Training Dataset: Likely based on public datasets (e.g., VGGFace2, CASIA-WebFace), though Davis King hasn't disclosed full details.

Embedding Output: maps each face image to a 128-dimensional vector. This is called a face embedding.

Distance Matching: Recognition is done by comparing distances between embeddings.

What is Dlib used for?

Feature	Description
🧍 Face detection	HOG and CNN-based face detectors
📍 Facial landmark detection	5-point or 68-point landmarks
🧠 Face recognition	128D embeddings using ResNet-style CNN
🧮 ML algorithms	SVM, k-NN, k-Means, decision trees
🧹 Image processing	Filtering, resizing, color conversions
📊 Numerical optimization	Fast solvers for regression and classification

Dlib For Face Recognition

Input: RGB image of a detected and aligned face (usually 150x150 or 128x128 pixels)

Output: A 128-dimensional vector (face embedding)

What happens at inference?
1. Detect face: Using HOG or CNN
2. Align face: Based on landmarks
3. Resize and preprocess
4. Pass through ResNet: Get 128D embedding
5. Compare embeddings: Use Euclidean distance

Dlib Face Detection can use HOG or CNN

For Face Detection Dlib can use:
HOG (Histogram of Oriented Gradients) – fast, CPU-based.
CNN – slower, more accurate, especially for tilted/partial faces.

Here is the modification in the script to use CNN in the detection process :

```
face_recognition.face_locations(image, model="hog" or "cnn")
```

Nevertheless for Face Recognition Dlib always uses the Dlib-trained CNN model (not HOG) for generating embeddings.

Dlib Model Advantages for Face Recognition

Easy to use with Python (pip install dlib)

High-performance C++ backend

Lightweight and doesn't require big frameworks like TensorFlow or PyTorch

Pure CPU-compatible, though some features can be GPU-accelerated

Pre-trained - Ready to use, no training needed

Accurate Very high recognition accuracy

Compact output - 128D vector is efficient for storage

Tuned for faces - Specially optimized for faces

FaceNet

FaceNet is a Open Source deep convolutional neural network developed by Google researchers (Schroff et al., 2015). It introduced a powerful way to recognize faces using embeddings.

Key Concepts:
Embeddings: FaceNet maps each face image to a 128-dimensional vector.
Triplet Loss: It trains on anchor-positive-negative triplets so that embeddings of the same person are close, and different people are far apart in vector space.
Pre-trained Models: Implementations available via TensorFlow, Keras, PyTorch, etc.

Triplet Loss Function

FaceNet doesn't train with labels like person A, person B. Instead, it uses triplets:
Anchor: image of a person
Positive: another image of the same person
Negative: image of a different person

The goal is

```
||f(anchor) - f(positive)||² + α < ||f(anchor) - f(negative)||²
```

Where:

- `f(x)` = 128D embedding of image `x`
- `α` = margin (a small constant, like 0.2)

Applications

Face verification: Are these two faces the same person?

Face clustering: Group similar faces together

Face authentication: Secure login via facial identity

Face identification: Find person X from a database

Implementation Options

Official Google version (TensorFlow) – Not publicly released

OpenFace – PyTorch-based open-source FaceNet

FaceNet Keras – Community versions on GitHub

InsightFace – Modern and high-performance derivatives

DeepFace – Easy-to-use Python lib that wraps FaceNet and others

FaceNet vs Dlib

Davis King has stated that the training process was inspired by FaceNet and uses a similar philosophy.

Aspect	FaceNet	Dlib Face Recognition
Loss Function	Triplet Loss	Triplet-like or contrastive
Embedding Size	128D	128D
Backbone	Inception or ResNet	ResNet-34
Goal	Embedding for recognition	Embedding for recognition
Euclidean distance used	✅	✅
Open training code	Yes (FaceNet clones)	❌ (Dlib model is pre-trained)

FaceNet code example using TensorFlow and keras-facenet

Install keras-facenet and scipy libraries:

```
pip install keras-facenet
pip install scipy

from keras_facenet import FaceNet
import numpy as np
import cv2
import os

# Initialize the FaceNet embedder
embedder = FaceNet()

# Load and encode known image
def get_embedding(img_path):
    img = cv2.imread(img_path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    return embedder.embeddings([img])[0]  # returns a list
```

FaceNet code example using TensorFlow and keras-facenet

```
# Compare two images
def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

# Load known face
known_embedding = get_embedding("..\Images\Faces\Koko\koko_6.jpg")

# Simulate unknown face (could be webcam frame or file)
test_embedding = get_embedding("..\Images\Faces\Koko\koko_4.jpg")  # change to your test image

# Compare
similarity = cosine_similarity(known_embedding, test_embedding)

print(f"Cosine similarity: {similarity:.4f}")
if similarity > 0.6:
    print("Same person!")
else:
    print("Different person.")
```

Exercise

Propose a code to run face recognition from a webcam using keras-facenet for multiple people.
Compare the performance of FaceNet and Dlib

In this session we have covered:

Face Recognition with OpenCV & Deep Learning
Understanding FaceNet and Dlib face recognition models.
Real-time face recognition using OpenCV

In the next session we will covered:

Final Project – Building a Computer Vision Application
Choosing a project.
Data collection, processing, and augmentation.
Training a model using OpenCV and Deep Learning.

THANKS !!!